

The *-juxt* function takes a list of functions and returns a function with the juxtaposition of those functions. In the following example, $2 + 5$ is evaluated as part of the $+$ function, and then $2 - 5$ is evaluated as part of the $-$ function:

```
(-juxt functions) ;; Syntax
```

```
(funcall (-juxt '+ '-') 2 5)
(7 -3)
```

The *-compose* function accepts a list of functions and returns a function with the composition of those functions. In the example given below, the arguments 4 and 3 are added together to get 7, and then squared to get the result 49.

```
(-compose functions) ;; Syntax
```

```
(funcall (-compose 'square '+) 3 4)
49
```

The *-applify* function takes a function that modifies a *n-arity* function to a *1-arity* function. For example:

```
(-applify function) ;; Syntax
```

```
(-map (-applify '+) '((1 2 3) (4 5 6)))
(6 15)
```

You can use the *-on* function with an operator and a transformer function on a list of lists. In the following example, the length of the lists is computed and then they are sorted in increasing order.

```
(-on operator function) ;; Syntax
```

```
(-sort (-on '<' 'length) '((1 2) (1) (1 2 3)))
((1) (1 2) (1 2 3))
```

The *-flip* function swaps the order of arguments and evaluates the input function. For example:

```
(-flip function) ;; Syntax
```

```
(funcall (-flip '<) 4 3)
t
```

```
(funcall (-flip '-') 4 3)
-1
```

The *const* function takes a value and returns it, ignoring the other arguments passed to it. A couple of examples are provided below:

```
(-const value) ;; Syntax
```

```
(funcall (-const 5) :a 3 "foo")
5
(-map (-const 2) '(1 2 3 4))
(2 2 2 2)
```

The *-not* function takes a predicate and returns a unary predicate function that returns *True* if the predicate returns nil. For example:

```
(-not predicate) ;; Syntax
```

```
(funcall (-not 'even?) 5)
t
```

If you want to perform an *OR* function with a list of unary predicates, then the *-orfn* function can be used. A couple of examples are shown below:

```
(-orfn predicates) ;; Syntax
```

```
(funcall (-orfn 'even? 'stringp) 4)
t
(funcall (-orfn 'stringp 'even?) 4)
t
```

The *-andfn* takes a list of unary predicates and returns *True* if all the predicates return non-nil. For example:

```
(-andfn predicates) ;; Syntax
```

```
(funcall (-andfn 'stringp 'even?) 4)
nil
```

The *-iteratefn* takes two arguments — a function and a count. It composes the function a count number of times. In the following example, the *cdr* function is applied twice on the input list (1 2 3 4) and it returns the list (3 4).

```
(-iteratefn function count) ;; Syntax
```

```
(funcall (-iteratefn 'cdr 2) '(1 2 3 4))
(3 4)
```

You are encouraged to explore the other dash.el functions from its GitHub README file at <https://github.com/magnars/dash.el>. 

 By: Shakthi Kannan

The author is a free software enthusiast and blogs at shakthimaan.com.